

ANALYSE FONCIÈRE DES MUTATIONS IMMOBILIÈRES

De la donnée brute au tableau de bord



Remerciements

Nous tenons à remercier M. Le Veler, notre professeur et encadrant pour ce projet. Ses explications claires et précises sur les consignes et les attentes concernant les livrables nous ont été d'une aide précieuse tout au long de ce travail. Grâce à sa pédagogie et à sa disponibilité, nous avons pu avancer sereinement et structurer notre démarche avec rigueur.

Table des matières

Introduction.....	3
1. Organisation du projet.....	3
A. Gestion de projet et répartition des rôles.....	3
B. Planning global et chronologie du projet.....	4
C. Outils utilisés.....	5
2. Contexte et compréhension des données.....	6
A. Origine et finalité des données.....	6
B. Structure et reconstitution des données.....	6
3. Analyse exploratoire des données.....	7
A. Description globale du dataset.....	7
B. Qualité des données.....	7
C. Dictionnaire de données.....	8
4. Modélisation des données.....	9
A. Modèle conceptuel (MCD).....	9
B. Modèle logique (MLD).....	9
C. Modèle en étoile.....	10
5. Mise en place de l'ETL et alimentation des données.....	11
A. Préambule à l'alimentation.....	11
B. Alimentation du MLD (CSV → MLD).....	12
C. Alimentation du modèle en étoile (MLD → DWH).....	14
6. Exploitation et visualisation des données.....	19
A. Chargement des données.....	19
B. Cahier de recettes.....	20
C. Tableau de bord.....	21
7. Difficultés rencontrées.....	23
A. Gestion de la volumétrie des données.....	23
B. Modélisation des données.....	23
C. Problématiques liées à la configuration et à l'automatisation.....	23
Conclusion.....	24
Bilans personnels.....	24
A. Anna.....	24
B. Justine.....	25

Introduction

Dans le cadre d'un projet d'analyse des valeurs foncières en France, nous avons travaillé sur la transformation de données brutes en un modèle structuré puis sur leur exploitation à travers un tableau de bord décisionnel. L'objectif était de construire une chaîne de traitement complète, depuis la donnée brute jusqu'à sa valorisation.

Pour ce projet, nous avons utilisé le dataset DVF (Demandes de Valeurs Foncières), disponible en open data. Il contient les transactions immobilières réalisées en France sur les dernières années, avec des informations sur les biens, leur localisation et leur valeur.

Le projet s'est déroulé en plusieurs étapes : compréhension et exploration des données, modélisation (MCD, MLD et modèle en étoile), mise en place d'un ETL pour automatiser les traitements puis création de visualisations avec Power BI. Cette démarche nous a permis de mieux comprendre les enjeux concrets d'un projet data, notamment en termes de structuration, de qualité des données et de performance.

1. Organisation du projet

A. Gestion de projet et répartition des rôles

Notre projet s'est déroulé du 6 février 2026 au 12 juin 2026, avec une équipe de deux personnes : Justine et Anna.

Anna a assuré le rôle de cheffe de projet sur cette partie, en plus de la gestion de l'alimentation des données. Justine était cheffe de projet sur les parties datamining et statistiques et a travaillé sur l'analyse exploratoire, la modélisation. Ensemble nous avons travaillé sur la mise en place du projet, la datavisualisation et la rédaction de ce rapport.

Concernant notre méthodologie de travail, nous faisons des points hebdomadaires pour faire le point sur l'avancement, les blocages et les prochains livrables. Nous avons également utilisé Trello pour répartir les tâches par livrable et suivre l'avancement en temps réel. Nous notions également les problèmes rencontrés, les durées prévisionnelles et réelles et les décisions prises.

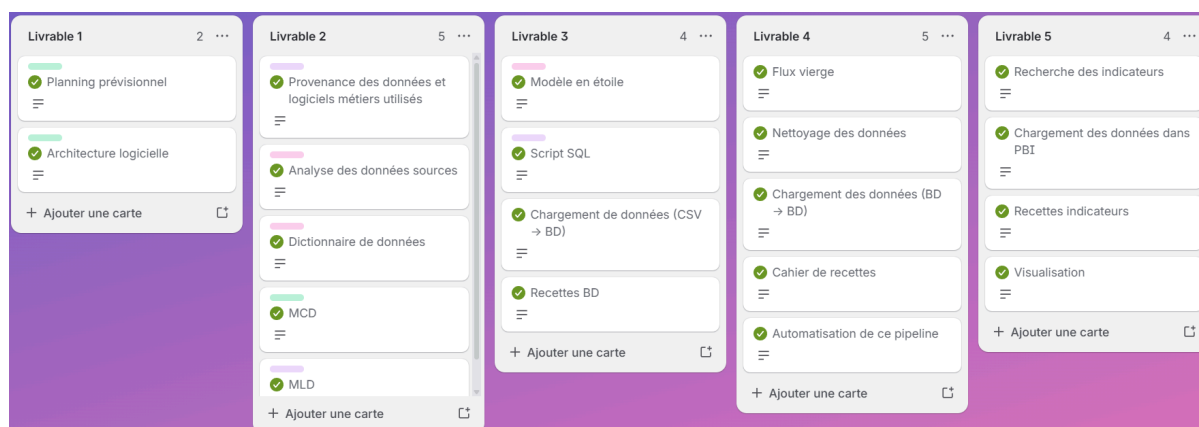


Figure 1 : Capture d'écran de notre Trello

Lien : <https://trello.com/b/O0ZMMIQ1/mon-tableau-trello>

B. Planning global et chronologie du projet

Le tableau ci-dessous synthétise le planning global du projet, avec les temps estimés, les temps réels et les responsables.

Tâche	Temps estimé (h)	Temps réel (h)	Fait par
Découverte du sujet	1	1	Anna & Justine
Analyse exploratoire	3	3	Justine
Modélisation des données (MCD, MLD, modèle en étoile)	3	5	Justine
Alimentation du MLD	4	12	Anna
Alimentation du modèle en étoile	6	7	Anna
Datavisualisation	5	6	Anna & Justine
Rédaction du rapport	6	6	Anna & Justine
Total	28	40	

Tableau 1 : Planning du projet selon les différentes tâches

Le principal décalage concerne l'alimentation du MLD (12h au lieu de 4h). En réalité, cette tâche était plus complexe qu'initialement prévu : on pensait simplement importer les données CSV dans une seule table alors qu'il a fallu les répartir dans plusieurs tables du MLD en respectant les relations.

Pour visualiser l'avancement et les dépendances entre tâches, nous avons utilisé un diagramme de Gantt simplifié.



Figure 2 : Diagramme de Gantt du projet

C. Outils utilisés

Gestion de projet

Trello pour répartir les tâches par livrable, suivre l'avancement, prioriser les actions et noter en détail ce qui a été fait, les problèmes rencontrés et ce qu'il reste à faire.

Modélisation des données

Looping car il permet de passer du MCD au MLD et de générer automatiquement les scripts SQL de création de tables.

Base de données

Oracle + SQL Developer car nous le maîtrisons bien (utilisé en cours et en alternance). Il est très bien pour stocker et gérer les données.

ETL

Apache Hop car c'est un ETL open source et l'une de nous l'utilise déjà en alternance. Cette expérience nous a permis d'aller plus vite dans la mise en place des traitements.

Automatisation de l'ETL

Script .bat pour automatiser l'exécution des workflows Apache Hop.

Datavisualisation

Power BI car nous connaissons bien l'outil (cours et projets précédents). Il est gratuit et puissant pour créer des tableaux de bord interactifs

2. Contexte et compréhension des données

A. Origine et finalité des données

Le dataset utilisé, *Demandes de valeurs foncières géolocalisées*, est une version enrichie du dataset *Demandes de valeurs foncières (DVF)* publié sur la plateforme open data de l'État, data.gouv.fr.

Ces données, produites par la Direction générale des Finances publiques, proviennent des actes de vente enregistrés par les notaires lors des transactions immobilières. Après la signature d'une vente, les informations sont transmises à l'administration fiscale via les services de publicité foncière puis intégrées dans la base DVF.

Les données incluent les transactions immobilières en France (maisons, appartements, dépendances, locaux industriels et commerciaux) des cinq dernières années et sont publiées en open data pour améliorer la transparence du marché immobilier.

Le dataset DVF permet de mieux comprendre le marché immobilier en France. Il peut être utilisé par des particuliers pour estimer la valeur d'un bien, par des chercheurs ou étudiants pour faire des analyses ou par des collectivités pour étudier l'évolution du foncier.

B. Structure et reconstitution des données

Le dataset est fourni sous forme de fichiers texte ou CSV. Chaque ligne correspond à une parcelle vendue (une transaction pouvant regrouper plusieurs parcelles). On y retrouve des informations comme la date de vente, la valeur foncière, le type de bien, la surface, la commune, le code postal, ainsi que des coordonnées géographiques (latitude et longitude).

Même si les données sont diffusées sous forme de fichiers plats, elles proviennent à l'origine d'une base relationnelle. Celle-ci est probablement composée de plusieurs tables (transactions, biens, parcelles, localisation, temps) qui sont ensuite regroupées pour produire le dataset final.

3. Analyse exploratoire des données

A. Description globale du dataset

Le dataset contient 20 102 739 observations et 40 variables, couvrant les transactions immobilières en France entre 2020 et 2025. Chaque ligne correspond à une parcelle concernée par une mutation et une même transaction peut apparaître sur plusieurs lignes si elle implique plusieurs parcelles ou lots. La variable `id_mutation` permet de regrouper ces lignes par transaction.

Les variables peuvent être regroupées en plusieurs dimensions principales :

- Informations sur la transaction (`date_mutation`, `nature_mutation`, `valeur_fonciere`)
- Localisation géographique (`code_postal`, `nom_commune`, `latitude`, `longitude`)
- Caractéristiques du bien (`type_local`, `surface_reelle_bati`, `nombre_pieces_principales`)
- Informations sur les parcelles (`id_parcelle`, `surface_terrain`, `nature_culture`)
- Informations sur les lots de copropriété (`lot1_numero`, `lot1_surface_carrez`, `nombre_lots`)

B. Qualité des données

La variable `valeur_fonciere` est presque entièrement renseignée, avec seulement 0,97 % de valeurs manquantes et les montants varient de quelques centimes à plus de 14 milliards d'euros, reflétant des transactions exceptionnelles ou regroupant plusieurs biens. Les informations de localisation sont elles aussi fiables, avec moins de 2,1 % de valeurs manquantes pour les coordonnées.

Certaines variables sont moins renseignées, notamment celles concernant les surfaces `carrez` ou certaines caractéristiques du bâti car elles ne concernent que certains types de biens. Le code type de local montre que la majorité des biens sont des dépendances (42 %), suivis des maisons (28,3 %) et appartements (24,2 %) et les locaux industriels et commerciaux reste minoritaire (5,4 %).

Variable	Type	NA	Vide	Manquant	%	Unique	Min	Max
<code>id_mutation</code>	character	0	0	0	0	7338623		
<code>date_mutation</code>	IDate	0	0	0	0	1806		
<code>numero_disposition</code>	integer	0	0	0	0	1204	1	1 246
<code>nature_mutation</code>	character	0	0	0	0	6		
<code>valeur_fonciere</code>	numeric	194390	0	194390	0,97	466668	0,01	14 150 000 000
<code>adresse_numero</code>	integer	7474511	0	7474511	37,18	9127	1	9 999
<code>adresse_suffixe</code>	character	0	19197659	19197659	95,5	41		
<code>adresse_nom_voie</code>	character	0	151263	151263	0,75	1256963		
<code>adresse_code_voie</code>	character	0	150482	150482	0,75	20065		
<code>code_postal</code>	integer	151600	0	151600	0,75	5872	1000	97 490
<code>code_commune</code>	character	0	0	0	0	33380		
<code>nom_commune</code>	character	0	0	0	0	31170		

Tableau 2 : Extrait du tableau d'analyse des données sources

C. Dictionnaire de données

Le dictionnaire des données présente l'ensemble des variables contenues dans le dataset *Demandes de valeurs foncières géolocalisées*. Pour chaque variable, il indique :

- Description : le rôle ou la signification de la variable.
- Type : le format de la donnée (numérique, caractère, date...).
- Unité : si applicable, l'unité de mesure (m², €...).
- Exemple : une valeur représentative pour illustrer la variable.
- Remarque : des précisions utiles pour l'interprétation ou le contexte.

Les variables couvrent différents aspects des transactions immobilières : identification de la mutation, caractéristiques du bien, localisation géographique et administrative, informations sur les parcelles et copropriétés. Le dictionnaire de données complet est disponible en annexe. L'extrait ci-dessous détaille chacune de ces variables avec les informations correspondantes pour faciliter la compréhension et l'utilisation du dataset.

Variable	Description	Type	Unité	Exemple	Remarque
id_mutation	Identifiant de mutation	character		44013	Identifiant unique d'une transaction immobilière
nature_mutation	Nature de la mutation	character		Vente	Type de transaction (vente, échange, adjudication...)
adresse_suffixe	Suffixe du numéro de l'adresse	character		1 A	Complément du numéro d'adresse
adresse_nom_voie	Nom de la voie de l'adresse	character		SAINT JULIEN	Nom de la rue
adresse_code_voie	Code FANTOIR de la voie	character		B064	Code officiel de la voie (référentiel FANTOIR)
code_commune	Code commune INSEE	character		1367	Identifiant officiel de la commune
nom_commune	Nom de la commune	character		Lille	Nom de la commune
code_departement	Code département INSEE	character		1	Code du département
ancien_nom_commune	Ancien nom de la commune	character		Sannerville	Utilisé en cas de fusion ou changement de commune
id_parcelle	Identifiant de parcelle	character		013670000A0008	Identifiant cadastral de la parcelle

Tableau 3 : Extrait du dictionnaire de données

4. Modélisation des données

A. Modèle conceptuel (MCD)

Nous avons étudié les données pour identifier les principales entités et les relations entre elles. Le MCD obtenu ci-dessous met en évidence les entités principales du jeu de données comme Mutation, Parcelle, Adresse et leurs associations.

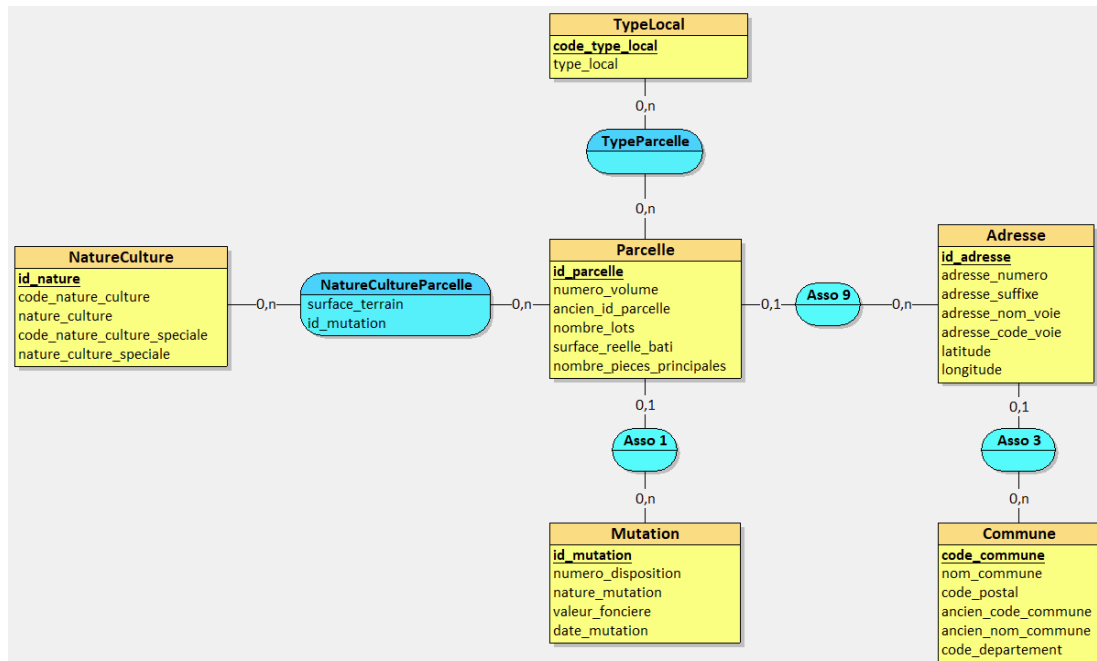


Figure 3 : MCD

B. Modèle logique (MLD)

Le MCD a ensuite été converti en MLD directement à l'aide de Looping. L'outil permet de générer automatiquement le modèle logique correspondant. Cette conversion a également permis de normaliser la structure des données et de définir les clés primaires et étrangères.

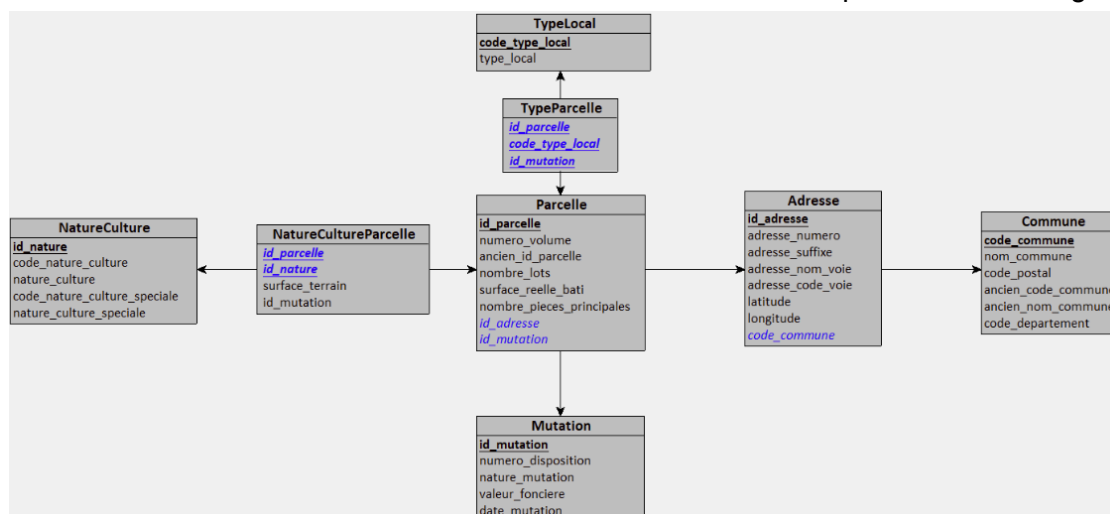


Figure 4 : MLD

Le script SQL de création de ces tables est disponible en annexe.

C. Modèle en étoile

Les données ont été regroupées en dimensions selon leur nature afin de structurer l'analyse autour de axes cohérents.

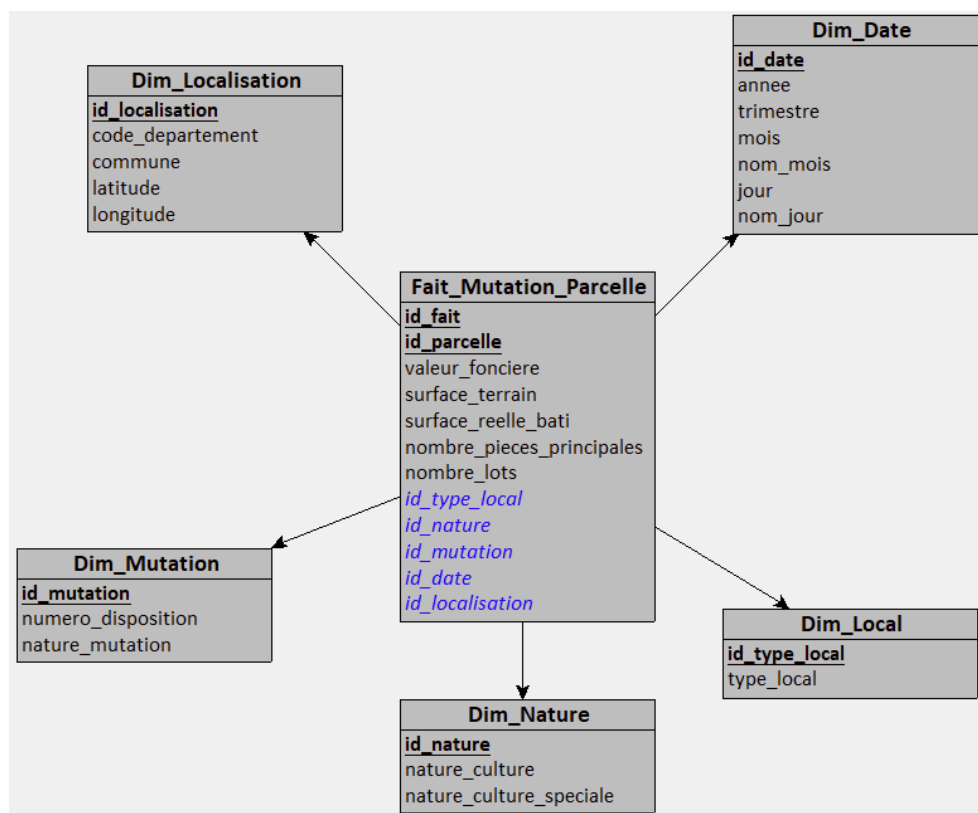


Figure 5 : Modèle en étoile

- La dimension dim_date regroupe toutes les informations temporelles (année, mois, trimestre, jour) pour étudier les évolutions dans le temps.
- La dimension dim_localisation rassemble les informations géographiques (département, commune, latitude et longitude) pour analyser la répartition spatiale des ventes.
- Les dimensions dim_local et dim_nature donnent les caractéristiques descriptives du bien (type de local et nature de culture) permettant d'analyser l'impact des propriétés du bien sur sa valeur foncière.
- Enfin, la dimension dim_mutation contient la nature des transactions pour distinguer les types de mutations immobilières.

Cette structuration par thèmes nous a permis de préparer les données, garantissant une meilleure lisibilité des données et une exploitation efficace en analyse multidimensionnelle, avant leur chargement dans PowerBI, notre outil de datavisualisation.

5. Mise en place de l'ETL et alimentation des données

A. Préambule à l'alimentation

Nous avons créé deux nouvelles connexions Oracle. Une correspondant au MLD (MUTATIONS_SOURCE) et une autre au modèle en étoile (MUTATIONS_CIBLE).

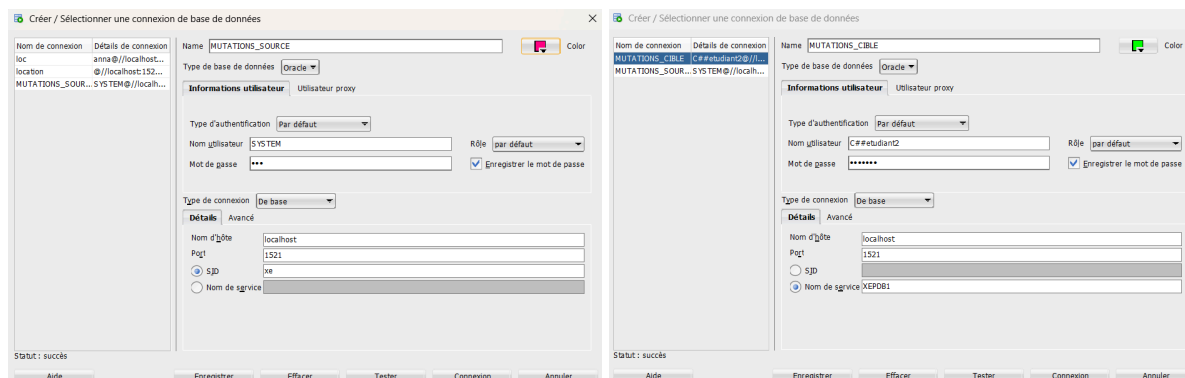


Figure 6 : Création et paramétrage des connexions Oracle dans SQL Developer

Ces deux connexions ont ensuite été intégrées et configurées dans l'ETL Hop pour permettre l'extraction, la transformation et le chargement des données entre les deux modèles.

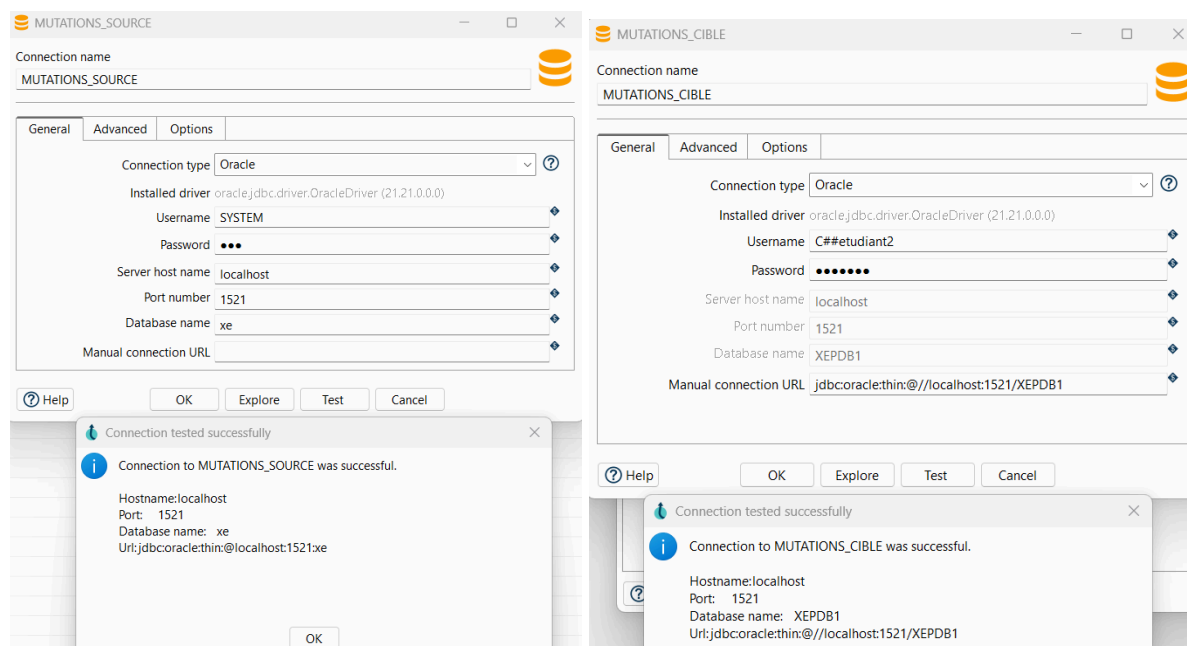


Figure 7 : Intégration des connexions dans Hop

Nous avons ensuite créé les tables à partir de nos scripts SQL (voir annexe).

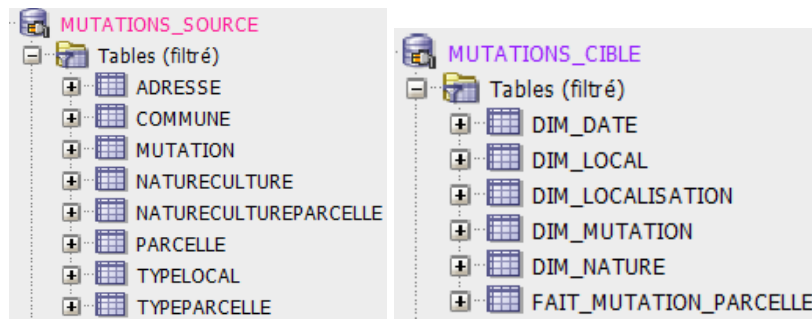


Figure 8 : Tables de notre MLD (à gauche) et modèle en étoile (à droite)

B. Alimentation du MLD (CSV → MLD)

L'objectif est d'intégrer les données issues du fichier csv dans la base de données relationnelle, en respectant la structure définie dans le MLD.

a. Flux de traitement

Pour insérer les données, plusieurs étapes sont mises en place dans le flux ETL :

- Extraction : récupération des données et définition des types
- Sélection : choix des champs utiles pour le traitement et l'insertion
- Filtrage : suppression des lignes avec une clé primaire nulle
- Tri : tri des données selon la clé primaire
- Déduplication : suppression des doublons
- Insertion : insertion des données dans la table cible



Figure 9 : Exemple du flux permettant d'alimenter la table COMMUNE

Certains champs ont nécessité des transformations pour s'adapter au modèle. Par exemple, la table NATURECULTURE n'avait pas de clé primaire unique. Nous avons donc créé un identifiant `id_nature` en concaténant `code_nature_culture` et le `code_nature_culture_speciale` pour garantir l'unicité des lignes.

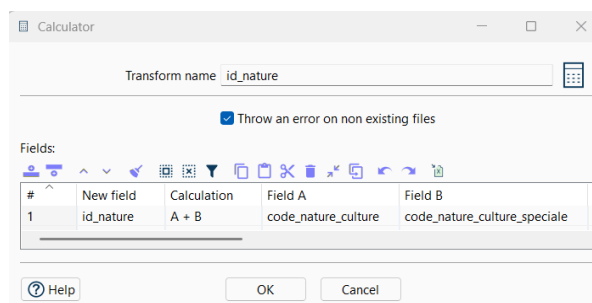


Figure 10 : Etape de concaténation dans Hop pour créer un identifiant `id_nature`

Nous avons fait une étape similaire pour créer `id_adresse` qui concatène `adresse_numero`, `adresse_nom_voie` et `code_commune`.

b. Orchestration

Pour automatiser et fiabiliser le processus d'intégration, un workflow d'orchestration a été mis en place pour exécuter les 8 pipelines ETL de manière séquentielle pour respecter les dépendances entre les tables. Ce workflow garantit le bon enchaînement des traitements (extraction, transformation et chargement) pour chaque table du MLD.

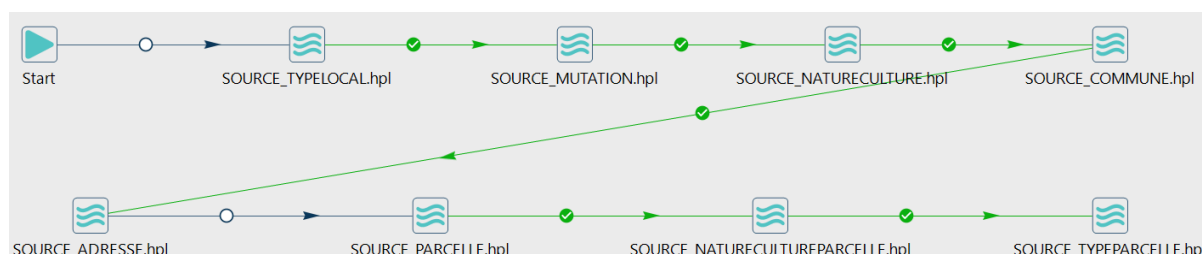


Figure 11 : Workflow Source

c. Cahier de recettes

Pour vérifier la bonne intégration des données, nous avons réalisé des tests de validation. D'une part, un script Python a permis de calculer le nombre de valeurs distinctes pour certains champs. D'autre part, nous avons vérifié que ce nombre correspondait au nombre de lignes présentes dans les tables en base.

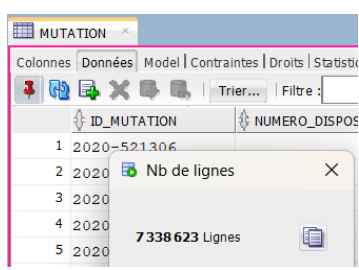
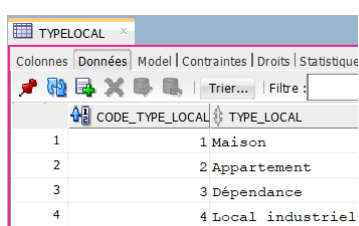
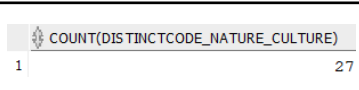
Champ (fichier csv) / Table en base	Nb modalités fichier csv	Nb lignes table	Validation
id_mutation / MUTATION	id_mutation : 7338623		✓
code_type_local / TYPELOCAL	code_type_local : 4		✓
code_nature_culture / NATURECULTURE	code_nature_culture : 27		✓
code_nature_culture_ speciale / NATURECULTURE	code_nature_culture_speciale : 139		✓

Tableau 4 : Cahier de recettes entre le fichier CSV et Oracle

C. Alimentation du modèle en étoile (MLD → DWH)

Nous avons mis en place un processus ETL pour alimenter le modèle en étoile à partir du MLD. Ce processus se décompose en deux types de flux : les tables de dimensions et la table de faits.

a. Dimensions

L'alimentation des tables de dimensions se fait en deux étapes principales :

- Extraction : extraction des données depuis les tables du MLD (schéma source)

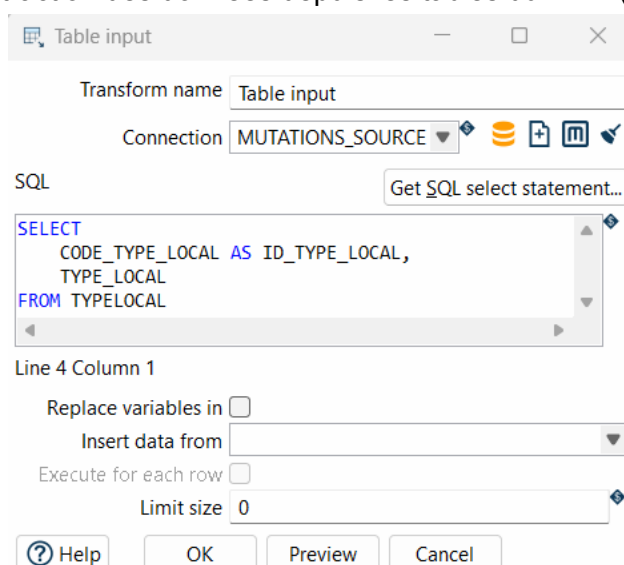


Figure 12 : Exemple d'une étape Table input pour la table cible DIM_LOCAL

- Insert / update : insertion des nouvelles données ou mise à jour des données existantes dans les tables de dimensions du modèle en étoile

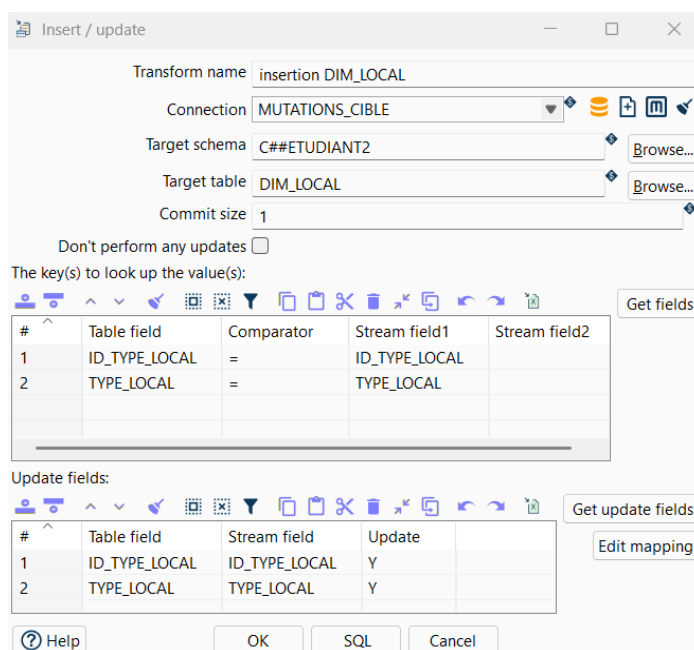


Figure 13 : Exemple d'une étape Insert / update pour la table cible DIM_LOCAL

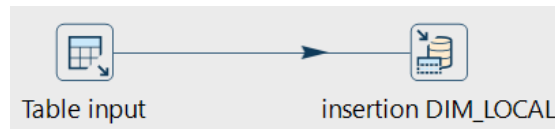


Figure 14 : Exemple d'un flux ETL pour l'alimentation de la table DIM_LOCAL

b. Table de faits

L'alimentation de la table de faits FAIT_MUTATION_PARCELLE repose sur le même principe général (extraction puis insertion) mais avec quelques spécificités :

D'une part, la requête SQL utilisée dans l'étape Table Input est plus complexe car elle nécessite de croiser et joindre plusieurs tables du MLD.

Transform name: Table input

Connection: MUTATIONS_SOURCE

SQL

```

SELECT
  p.id_parcelle,
  m.valeur_fonciere,
  ncp.surface_terrain,
  p.surface_reelle_bati,
  p.nombre_pieces_principales,
  p.nombre_lots,
  tp.code_type_local as id_type_local,
  ncp.id_nature,
  m.id_mutation,
  m.date_mutation as id_date,
  c.CODE_DEPARTEMENT || c.NOM_COMMUNE AS id_localisation
FROM Parcelle p
LEFT JOIN Mutation m
ON p.id_mutation = m.id_mutation
LEFT JOIN NatureCultureParcelle ncp
ON p.id_parcelle = ncp.id_parcelle AND p.id_mutation = ncp.id_mutation
LEFT JOIN TypeParcelle tp
ON p.id_parcelle = tp.id_parcelle AND p.id_mutation = tp.id_mutation
LEFT JOIN Adresse a
ON p.id_adresse = a.id_adresse
LEFT JOIN Commune c
ON a.code_commune = c.code_commune
  
```

Line 1 Column 1

Replace variables in script ☐

Insert data from transform

Execute for each row ☐

Limit size 0

Buttons: Help, OK, Preview, Cancel

Figure 15 : Etape Table input pour la table FAIT_MUTATION_PARCELLE

D'autre part, le volume de données étant beaucoup plus important, une insertion classique aurait été trop lente. Pour optimiser les performances, nous avons utilisé un Oracle Bulk Loader. Cette méthode permet de charger un grand volume de données en une seule opération. Elle est plus rapide qu'un Insert car les données sont insérées en masse et non ligne par ligne.

Cependant, contrairement à un Insert, le Bulk Loader ignore les lignes contenant des valeurs manquantes, notamment lorsque certaines clés de dimensions sont absentes (ex : type de local non renseigné).

Pour éviter la perte de données, nous avons ajouté une valeur par défaut dans chaque table de dimension. Par exemple, dans DIM_LOCAL, un id_type_local = -1 correspond à "inconnu". Ainsi, toutes les lignes, même incomplètes, peuvent être intégrées dans la table de faits.

ID_TYPE_LOCAL	TYPE_LOCAL
1	Maison
2	Appartement
3	Dépendance
4	Local industriel. commercial ou assim..
-1	Unknown

Figure 16 : Table DIM_LOCAL avec l'ID_TYPE_LOCAL -1

The figure consists of two screenshots of the Oracle Bulk Loader application. The left screenshot shows the 'Bulk loader' tab with the following configuration:

- Transform name: Oracle bulk loader FAIT_MUTATION_PARCELLE
- Connection: MUTATIONS_CIBLE
- Target schema: C##ETUDIANT2
- Target table: FAIT_MUTATION_PARCELLE
- Sqlldr path: C:\oracle\21c\dbhomeXE\bin\sqlldr.exe
- Load method: Automatic load (at the end)
- Load action: Append
- Maximum errors: 50
- Commit: 100000
- Bind Size: 0
- Read Size: 0
- Control file: C:\sqlldr\control0.cfg
- Data file: C:\sqlldr\data.csv
- Log file: C:\sqlldr\load.log
- Bad file: C:\sqlldr\load.bad
- Discard file: C:\sqlldr\load.dsc
- Encoding: (empty)
- Oracle Character Set: (empty)
- Alternate Record Terminator: (empty)
- Direct path: ☐
- Erase cfg/dat files after use: ☒
- Fail on warning: ☐
- Fail on error: ☒
- Load data in parallel: ☐

The right screenshot shows the 'Fields' tab with a mapping table:

#	Table field	Stream field
1	VALEUR_FONCIERE	VALEUR_FONCIERE
2	SURFACE_REELLE_BATI	SURFACE_REELLE_BATI
3	NOMBRE_PIECES_PRINCIPALES	NOMBRE_PIECES_PRINCIPALES
4	SURFACE_TERRAIN	SURFACE_TERRAIN
5	NOMBRE_LOTS	NOMBRE_LOTS
6	ID_TYPE_LOCAL	ID_TYPE_LOCAL
7	ID_MUTATION	ID_MUTATION
8	ID_NATURE	ID_NATURE
9	ID_LOCALISATION	ID_LOCALISATION
10	ID_PARCELLE	ID_PARCELLE
11	ID_DATE	ID_DATE

Figure 17 : Etape Oracle Bulk Loader pour alimenter la table FAIT_MUTATION_PARCELLE

c. Orchestration

Pour automatiser l'alimentation de nos tables du schéma cible (modèle en étoile), nous avons fait un flux workflow qui permet d'enchaîner les différents flux.

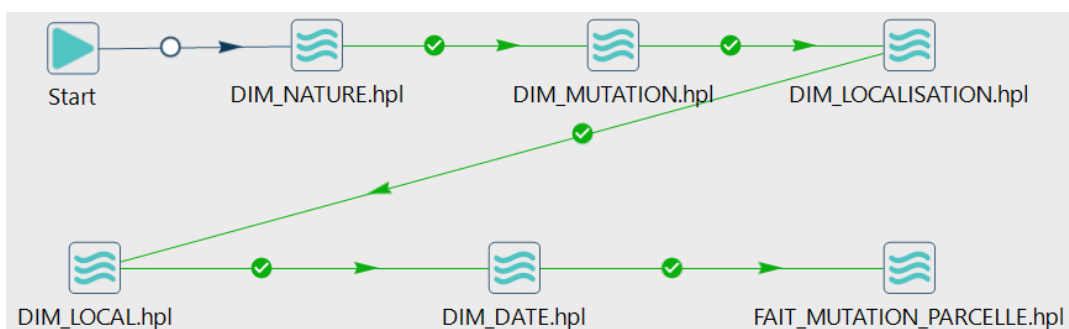


Figure 18 : Workflow cible

d. Cahier de recettes

Nous avons fait des tests pour vérifier la bonne alimentation des tables de dimension et de la table de faits.

	MLD (source)	Etoile (cible)	Validation
Mutations	<pre>SELECT count(id_mutation) as nb_source FROM MUTATION;</pre>	<pre>SELECT count(*) as nb_cible FROM DIM_MUTATION;</pre>	✓
Dates de mutations	<pre>SELECT count(distinct date_mutation) as nb_source FROM MUTATION;</pre>	<pre>SELECT count(*) as nb_cible FROM DIM_DATE;</pre>	✓
Types de locaux	<pre>SELECT count(*) as nb_source FROM TYPELOCAL;</pre>	<pre>INSERT INTO Dim_Local VALUES (-1, 'Unknown'); select count(*) from dim_local;</pre>	✓ *Nous avons rajouté un type local -1

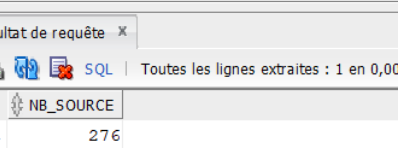
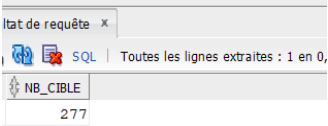
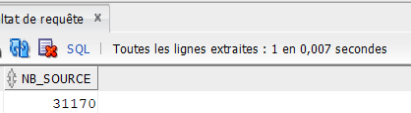
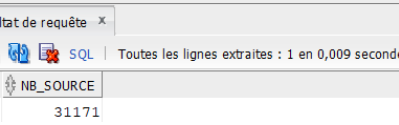
Natures de cultures	<pre>select count(id_nature) as nb_source from natureculture;</pre> 	<pre>INSERT INTO Dim_Nature VALUES ('UN', 'Unknown', 'Unknown'); select count(id_nature) as nb_cible from dim_nature;</pre> 	 *Nous avons rajouté une nature 'UN'
Communes	<pre>select count(distinct nom_commune) as nb_source from commune;</pre> 	<pre>INSERT INTO Dim_Localisation VALUES ('UNKNOWN', -1, 'Unknown', -1, -1); select count(distinct commune) as nb_source from dim_localisation;</pre> 	 *Nous avons rajouté une commune 'Unknown'

Tableau 5 : Cahier de recettes entre le MLD et le modèle en étoile

e. Automatisation de l'exécution

Pour éviter de devoir ouvrir l'interface Hop et de lancer manuellement le workflow, une procédure d'automatisation a été mise en place à l'aide d'un script batch (.bat) sous Windows.

```
@echo off

cd /d "C:\Users\annad\OneDrive\Documents\BUT SD\S6\CIBLE"

"C:\Program Files\hop\hop-run.bat" -f "C:\Users\annad\OneDrive\Documents\BUT SD\S6\CIBLE\workflow.hwf"

pause
```

Figure 19 : Script workflow.bat

Pour valider le bon fonctionnement de cette automatisation, un test a été réalisé en ajoutant deux nouvelles lignes dans la table de faits via le workflow. Le log d'exécution ci-dessous confirme que le script s'est déroulé sans erreur et que les données ont bien été intégrées à la base cible :

```
2026/05/31 19:21:30 - Oracle bulk loader.0 - OUTPUT>Table C##ETUDIANT2.FAIT_MUTATION_PARCELLE :
2026/05/31 19:21:30 - Oracle bulk loader.0 - OUTPUT> Chargement de 2 Lignes effectu?.
```

Figure 20 : Log d'exécution du script workflow.bat
(alimentation des tables dans le modèle en étoile)

Grâce à cette mise en place, le workflow peut désormais être déclenché d'un simple double-clic sur le fichier .bat. Il devient également possible de planifier son exécution automatique à intervalles réguliers via le Planificateur de tâches Windows (par exemple chaque nuit), garantissant une alimentation régulière et autonome de la base cible.

6. Exploitation et visualisation des données

A. Chargement des données

Nous avons d'abord établi une connexion à notre base de données Oracle en renseignant le serveur (localhost:1521/XEPDB1) ainsi que nos identifiants (utilisateur C##etudiant2).

Base de données Oracle

Serveur

Mode de connectivité des données ⓘ

☒ Importer

☐ DirectQuery

Figure 21 : Connexion à la base de données Oracle dans Power BI

Nous avons ensuite importé les 5 tables de dimension et la table de faits depuis notre schéma cible.

Navigateur

Options d'affichage

- localhost:1521/XEPDB1 [27]
 - ANONYMOUS
 - APPOSSYS
 - AUDSYS
 - C##ETUDIANT2 [8]
 - BIN\$321TVDDRM+DOxymfcu7Q+...
 - BIN\$EJQVG+9Qx6PGvZkEPHtg==50
 - ☒ DIM_DATE
 - ☒ DIM_LOCAL
 - ☒ DIM_LOCALISATION
 - ☒ DIM_MUTATION
 - ☒ DIM_NATURE
 - ☒ FAIT_MUTATION_PARCELLE
 - DBSFUSER
 - DBSNMP
 - DGPDB_INT
 - DIP
 - DVF
 - DVSY
 - GGSYS

Sélectionner les tables associées

FAIT_MUTATION_PARCELLE

ID_FAIT	ID_PARCELLE	VALEUR_FONCIERE	SURFACE_TERRAIN	SURFACE_REE
6316741	051330000A2345	320000.0		null
6316742	112620000K0424	108000.0		null
6316743	121450000AE028	274150.0		null
6316744	130010000C0941	310000.0		null
6316745	130280000C9334	540000.0		null
6316746	130560000AE0643	259000.0		null
6316747	130560000BC1651	120240.0		null
6316748	130870000AH0572	399000.0		null
6316749	130870000AH0572	369800.0		null
6316750	131170000AK0285	18000.0		null
6316751	132108590C0006	284000.0		null
6316752	132108590C0141	179011.5		null
6316753	132108590C0141	102212.8		null
6316754	132108590C0141	260905.0		null
6316755	132108590C0141	445171.0		null
6316756	132108590C0141	413145.0		null
6316757	132108590C0143	212156.0		null
6316758	143380000AE0159	18000.0		null
6316759	171420000D0208	275000.0		null
6316760	173850000AC0004	223700.0		null
6316761	240710000AB0043		null	null
6316762	276810000XC0573	185000.0		null
6316763	276810000XC0573	187239.0		null

Charger Transformer les données Annuler

Charger

- ☒ DIM_DATE
1 806 lignes depuis localhost:1521/XEPDB1.
- ☒ DIM_LOCAL
4 lignes depuis localhost:1521/XEPDB1.
- ☒ DIM_LOCALISATION
33 378 lignes depuis localhost:1521/XEPDB1.
- ☒ DIM_NATURE
276 lignes depuis localhost:1521/XEPDB1.
- ☒ DIM_MUTATION
7 338 623 lignes depuis localhost:1521/XEPDB1.
- ☒ FAIT_MUTATION_PARCELLE
6 316 417 lignes depuis localhost:1521/XEPDB1.

Annuler

Annuler

Figure 22 : Importation des tables de dimension et de faits

Le modèle obtenu dans Power BI correspond bien au modèle en étoile mis en place dans Oracle.

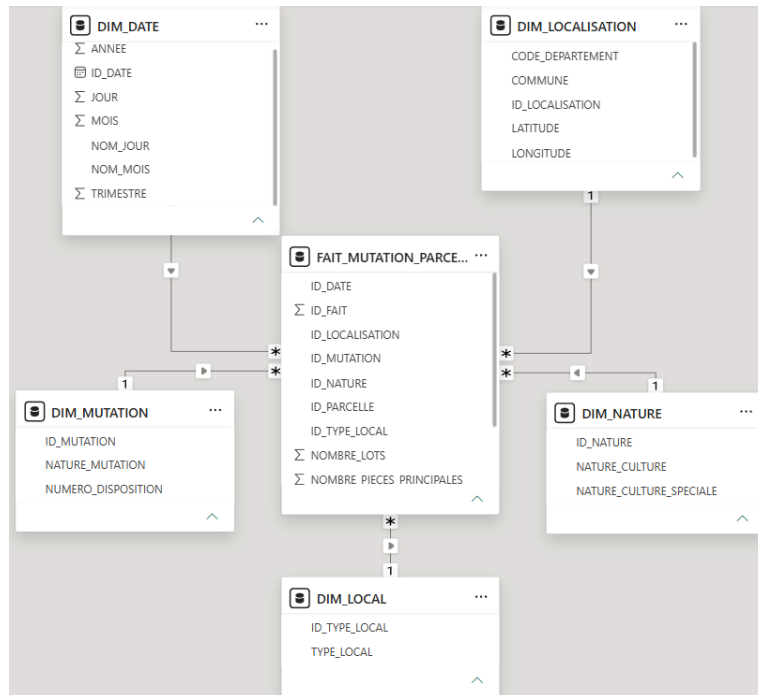


Figure 23 : Modélisation des données dans Power BI (modèle en étoile)

Après le chargement, nous avons fait quelques ajustements pour faciliter l'analyse, notamment en modifiant le type et le résumé de certains champs numériques (comme les identifiants ou l'année) pour éviter qu'ils soient interprétés comme des mesures.

B. Cahier de recettes

	Oracle (schéma cible)	PowerBI	Validation
Mutations	<pre>SELECT count(*) as nb_cible FROM DIM_MUTATION;</pre>	<pre>1 NB_MUTATIONS = FORMAT(2 COUNT(DIM_MUTATION[ID_MUTATION]), 3 "# ### ##0" 4)</pre>	✓
Dates de mutations	<pre>SELECT count(*) as nb_cible FROM DIM_DATE;</pre>	<pre>1 NB_DATES = FORMAT(2 COUNT(DIM_DATE[ID_DATE]), 3 "# ### ##0" 4)</pre>	✓
Types de locaux	<pre>select count(*) from dim_local;</pre>	<pre>1 NB_TYPES_LOCAUX = FORMAT(2 COUNT(DIM_LOCAL[ID_TYPE_LOCAL]), 3 "# ### ##0" 4)</pre>	✓

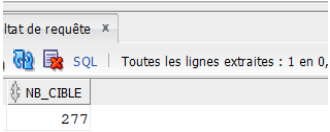
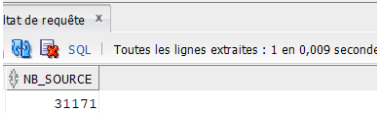
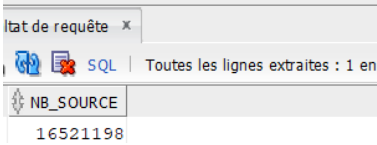
Natures de cultures	<pre>select count(id_nature) as nb_cible from dim_nature;</pre> 	<pre>1 NB_NATURES = FORMAT(2 COUNT(DIM_NATURE[ID_NATURE]), 3 "# ### #0") 4)</pre> 277 NB_NATURES	✓
Communes	<pre>select count(distinct commune) as nb_source from dim_localisation;</pre> 	<pre>1 NB_COMMUNES = 2 FORMAT(COUNTROWS(3 DISTINCT(DIM_LOCALISATION[COMMUNE]) 4), "# ### #0")</pre> 31 171 NB_COMMUNES	✓
Nombre de faits	<pre>select count(id_fait) as nb_source from fait_mutation_parcelle;</pre> 	<pre>1 NB_FAITS = FORMAT(2 COUNT(FAIT_MUTATION_PARCELLE[ID_FAIT]), 3 "# ### #0") 4)</pre> 16 521 198 NB_FAITS	✓

Tableau 6 : Cahier de recettes entre le modèle en étoile (Oracle) et Power BI

C. Tableau de bord

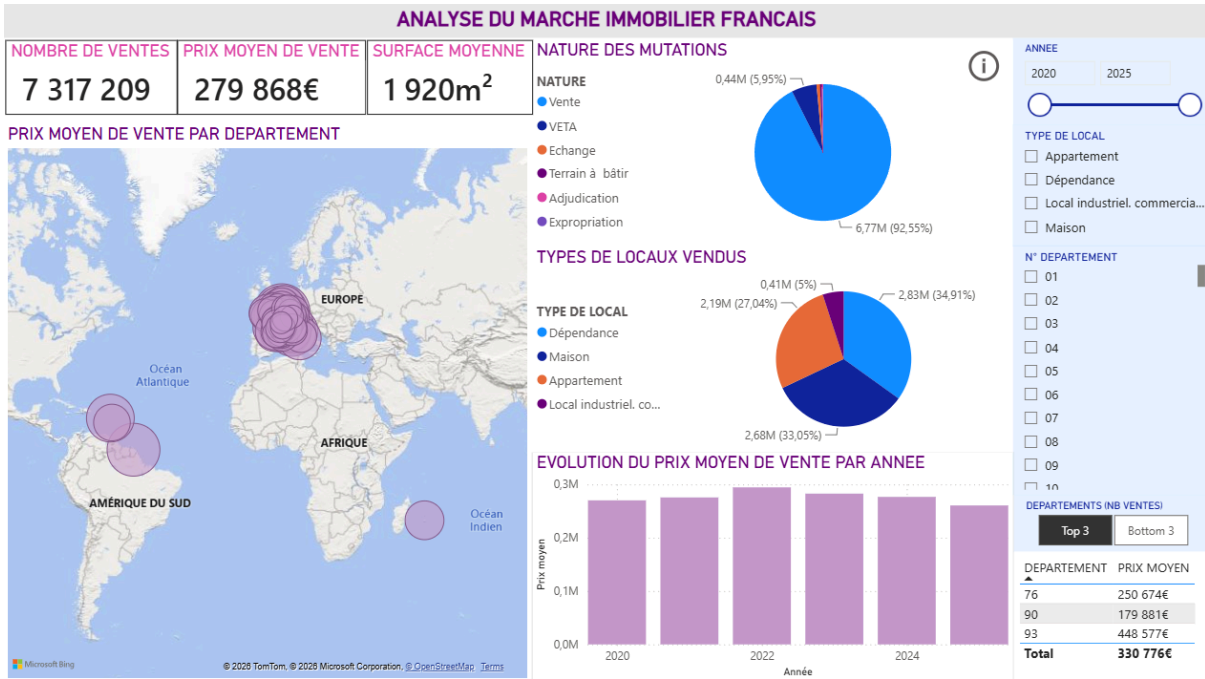
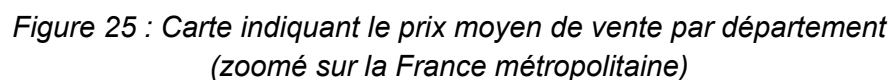


Figure 24 : Tableau de bord global

La carte occupe une place centrale dans l'analyse. Elle permet de visualiser la répartition géographique des prix moyens de vente par département. Chaque cercle représente un département et sa taille est proportionnelle au prix moyen observé. Cela permet d'identifier rapidement les zones où les prix sont les plus élevés, comme par exemple en région parisienne.



Enfin, le tableau de bord est interactif grâce à plusieurs filtres (année, type de bien, département). L'utilisateur peut ainsi affiner l'analyse selon ses besoins et explorer les données de manière plus précise.

7. Difficultés rencontrées

A. Gestion de la volumétrie des données

Le traitement des données a révélé des contraintes majeures liées à la taille des fichiers sources. En effet, le fichier CSV contenait plus de 22 millions de lignes, ce qui a considérablement ralenti les opérations de chargement et de transformation.

Par exemple, le chargement de 7 millions de lignes dans la table Mutation via l'ETL a nécessité 2 heures de traitement. Pour la table de faits, l'utilisation d'un simple Insert/update aurait entraîné des temps d'exécution de plusieurs jours, voire une semaine, rendant le processus inutilisable en l'état.

Comme solution mise en œuvre, nous sommes passées à un bulk loader Oracle pour réduire significativement les temps de traitement. Cependant, cette solution a nécessité une reconfiguration complète de l'infrastructure du schéma cible, notamment le passage de XE à XEPDB1 car le bulk loader ne fonctionne pas avec le même conteneur et également un paramétrage complexe du bulk loader.

B. Modélisation des données

Initialement perçue comme une étape simple, la modélisation s'est révélée plus complexe que prévu. L'identification de dépendances n-n imprévues a compliqué la construction du MCD, notamment en raison de l'unicité des identifiants de parcelle.

En effet, une même parcelle pouvait apparaître dans plusieurs mutations distinctes, ce qui a nécessité de multiples révisions de nos modèles (MCD, MLD et modèle en étoile). Malgré ces ajustements, nous considérons que notre modélisation reste perfectible. Si le projet était à refaire, nous aurions opté pour un identifiant unique en combinant systématiquement l'id_mutation et l'id_parcelle.

C. Problématiques liées à la configuration et à l'automatisation

Plusieurs difficultés techniques ont été rencontrées lors de la configuration des outils et de l'automatisation des processus.

- Problème de configuration dans SQL Developer

Les deux schémas créés étaient initialement associés au même utilisateur. SQL Developer les a donc considérés comme un seul et même schéma, ce qui a entraîné des liens indésirables entre les tables et perturbé la création et l'alimentation des tables. Une recherche approfondie a été nécessaire pour comprendre et corriger cette configuration.

- Automatisation de l'exécution avec Apache Hop

La création d'un fichier .bat pour automatiser l'exécution des workflows a posé des difficultés techniques. Cette étape a finalement été résolue grâce à l'aide d'un tiers extérieur, permettant de finaliser l'automatisation.

Conclusion

Cette partie nous a permis de créer en tableau de bord de bout en bout, en partant des données brutes jusqu'à la datavisualisation. Elle montre une démarche complète de data engineering, avec la modélisation, l'ETL et la valorisation des données.

La compréhension du dataset DVF a été essentielle pour orienter nos choix de modélisation, avec la création d'un MLD puis d'un modèle en étoile adapté à l'analyse. La mise en place de l'ETL avec Apache Hop a constitué le cœur technique du projet. Elle nous a permis de transformer des données volumineuses en un système structuré et automatisé, tout en prenant en compte les contraintes de performance et de qualité. L'intégration dans Power BI a ensuite permis de rendre ces données exploitables à travers un outil d'analyse.

Au final, nous avons réussi à construire une architecture data complète, malgré des difficultés liées à la volumétrie, à la modélisation et à l'automatisation.

Même si toutes les consignes ont été réalisées, certains points pourraient être améliorés. La modélisation pourrait être affinée, notamment pour mieux gérer les relations entre mutations et parcelles. De plus, le tableau de bord Power BI, bien que fonctionnel, pourrait être plus pertinent avec de vraies problématiques métier, afin de proposer des analyses plus ciblées et plus utiles.

Bilans personnels

A. Anna

a. Hard skills

- Modélisation des données

J'ai appris à concevoir des modèles (MCD, MLD, étoile) et à les ajuster en fonction des contraintes réelles. La gestion des dépendances n-n et l'identification des identifiants uniques ont été des défis qui m'ont permis de mieux comprendre l'importance d'une structure de données robuste.

- Flux ETL et gestion de volumétrie

J'ai travaillé sur des fichiers dépassant 22 millions de lignes, ce qui m'a amené à optimiser les traitements et les performances. J'ai exploré des solutions comme le bulk loader Oracle afin de réduire les temps d'exécution et de mieux appréhender les enjeux de performance en data engineering.

b. Soft skills

- Travail de groupe et autonomie

J'ai collaboré sur un projet data en partageant les tâches selon nos compétences et en choisissant collectivement les outils les plus adaptés. J'ai appris à confronter des idées, à communiquer efficacement et à trouver des compromis pour avancer.

- Remise en question

J'ai adopté une démarche itérative dans la modélisation des données. J'ai revu plusieurs fois nos choix (MCD, MLD, modèle en étoile) pour mieux répondre aux besoins réels du projet.

- Persévérance

J'ai surmonté des blocages techniques liés à la volumétrie et à la configuration de SQL Developer. J'ai testé différentes solutions, exploré plusieurs pistes et sollicité de l'aide lorsque nécessaire, notamment lors de l'alimentation de la table de faits.

B. Justine

a. Hard skills

- Analyse exploratoire des données

La réalisation de l'analyse exploratoire m'a permis de monter en compétence en programmation sous R et de développer ma capacité à analyser un jeu de données. Ce travail m'a permis d'identifier les principales caractéristiques des données ainsi que certaines anomalies, afin de mieux comprendre le jeu de données.

- Modélisation des données

Ce projet m'a permis d'approfondir mes compétences en modélisation décisionnelle à travers la conception du modèle en étoile. La structuration des données a nécessité une réflexion approfondie en raison de la multiplicité des mutations et des parcelles associées. J'ai ainsi appris à adapter un modèle aux contraintes métier tout en garantissant sa cohérence et sa pertinence pour l'analyse.

- Datavisualisation

La sélection des graphiques à intégrer au tableau de bord a demandé une réflexion particulière. Nous avons dû nous projeter dans les besoins métier afin de concevoir des visualisations adaptées aux objectifs d'analyse. Ce travail m'a permis de développer ma capacité à transformer des données complexes en informations claires et exploitables.

b. Soft skills

- Gestion de projet et organisation

La réalisation de ce projet en autonomie m'a permis de développer mes compétences en gestion de projet. La mise en place d'un planning prévisionnel, la définition des tâches à réaliser ainsi que le suivi de leur avancement nous ont aidées à respecter les échéances fixées.

- Communication et travail d'équipe

Travailler en binôme m'a appris à communiquer efficacement sur l'avancement du projet, à partager mes idées et à prendre en compte celles de ma partenaire. Les échanges réguliers ont permis de résoudre rapidement certaines difficultés et de prendre les décisions importantes concernant les méthodes et les outils utilisés ensemble.